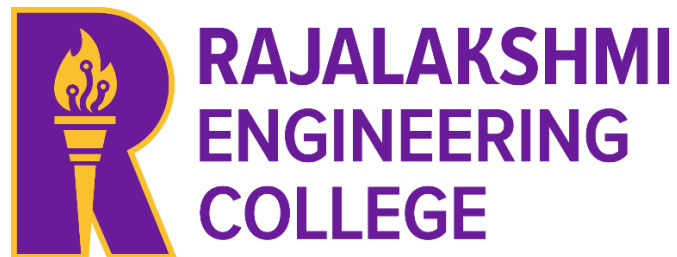


RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



AI23331 FUNDAMENTALS OF MACHINE LEARNING LABORATORY

Lab Manual

Name : R.PRIYANKA

Year / Branch / Section : 3rd Yr CSD

Register No. : 231701041

Semester : VI

Academic Year : 2025 -2026



RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: **R.PRIYANKA**

Academic Year: **2025-2026** Semester: **VI** Branch: **CSD**

Register No.

231701041

*Certified that this is the bonafide record of work done by the above student in
the (A123331) Fundamental Of Machine LearningLaboratory
during the academic year 2025- 2026*

Signature of Faculty in-charge

Submitted for the Practical Examination held on.....

Internal Examiner

External Examiner

INDEX

Reg. No. : 231701041 Name : R. PRIYANKA

Year : III Year / 3rd Year Branch : CSD – Computer Science and Design Sec : A

S. No.	Date	Title	Page No.	Teacher's Signature / Remarks
1		A Python Program To Implement Univariate Regression, Bivariate Regression And Multivariate Regression		
2		A Python Program To Implement Simple Linear Regression Using Least Square Method		
3		A Python Program To Implement Logistic Model		
4		A Python Program To Implement Single Layer Perceptron		
5		A Python Program To Implement Multi Layer Perceptron With Back Propagation.		
6		A Python Program To Do Face Recognition Using Svm Classifier.		
7		A Python Program To Implement Decision Tree.		
8		A Python Program To Implement Boosting.		
9		A Python Program To Implement Knn And K-Means		
10		A Python Program To Implement Dimensionality Reduction – PCA		

DATE:

REG NO: 231701041

EXP NO: 01

A PYTHON PROGRAM TO IMPLEMENT UNIVARIATE, BIVARIATE AND MULTIVARIATE REGRESSION

Aim:

Develop a Python program to implement univariate, bivariate, and multivariate regression models and analyze the relationship between dependent and independent variables.

Requirements :

Jupyter Notebook with Python, Iris.csv dataset, and required Python libraries

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

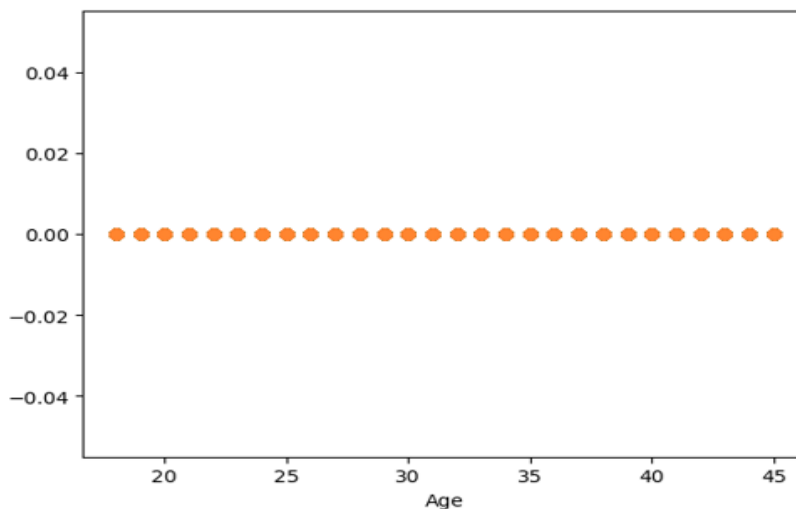
df = pd.read_csv('/mnt/data/Dry_Eye_Dataset.csv')
df.head()
df.shape # (20000, 26)

df_yes = df.loc[df['Dry Eye Disease'] == 'Y']
df_no = df.loc[df['Dry Eye Disease'] == 'N']
```

UNIVARIATE ANALYSIS:

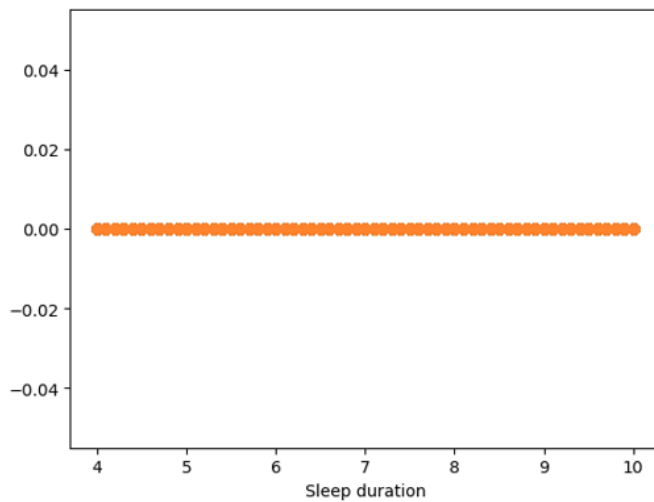
```
plt.scatter(df_yes['Age'], np.zeros_like(df_yes['Age']))
plt.scatter(df_no['Age'], np.zeros_like(df_no['Age']))
plt.xlabel('Age')
plt.show()
```

#age



```
# univariate for Sleep duration
```

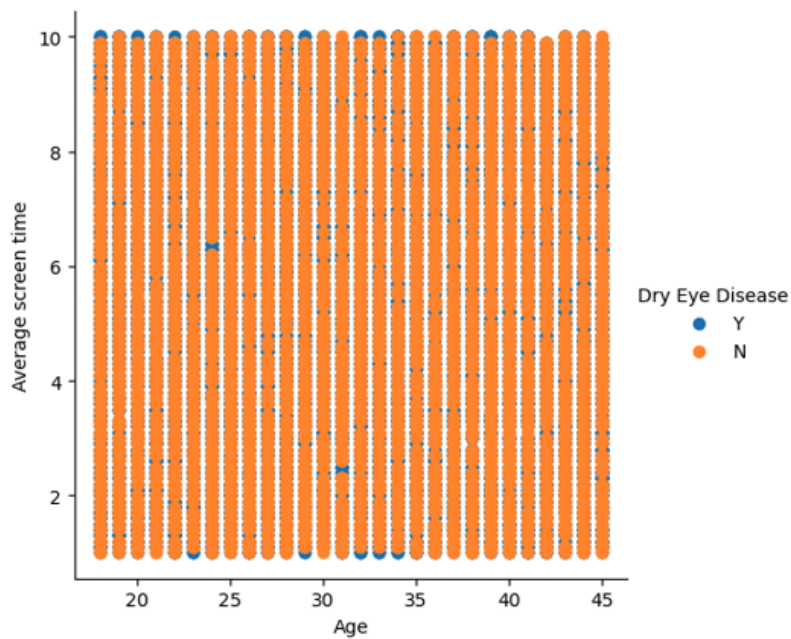
```
plt.scatter(df_yes['Sleep duration'], np.zeros_like(df_yes['Sleep duration']))  
plt.scatter(df_no['Sleep duration'], np.zeros_like(df_no['Sleep duration']))  
plt.xlabel('Sleep duration') plt.show()
```



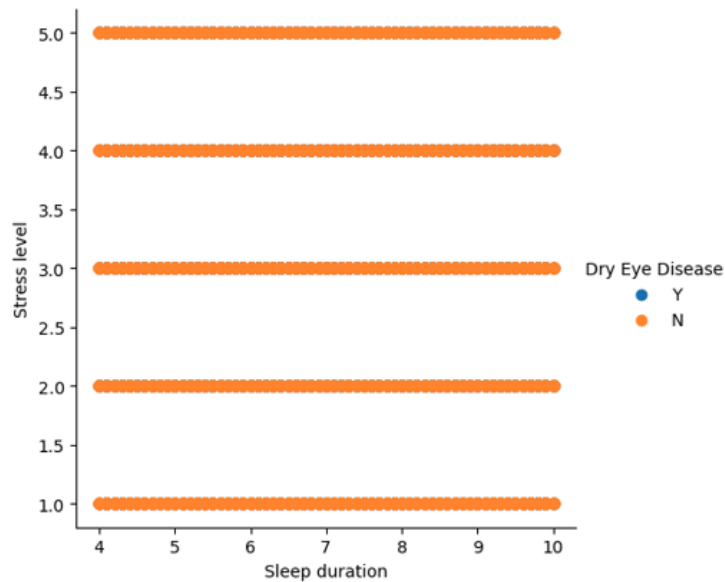
BIVARIATE ANALYSIS

```
sns.FacetGrid(df, hue='Dry Eye Disease', height=5)\  
  .map(plt.scatter, 'Age', 'Average screen time')\  
  .add_legend()
```

```
plt.show() #Age vs Screen time
```

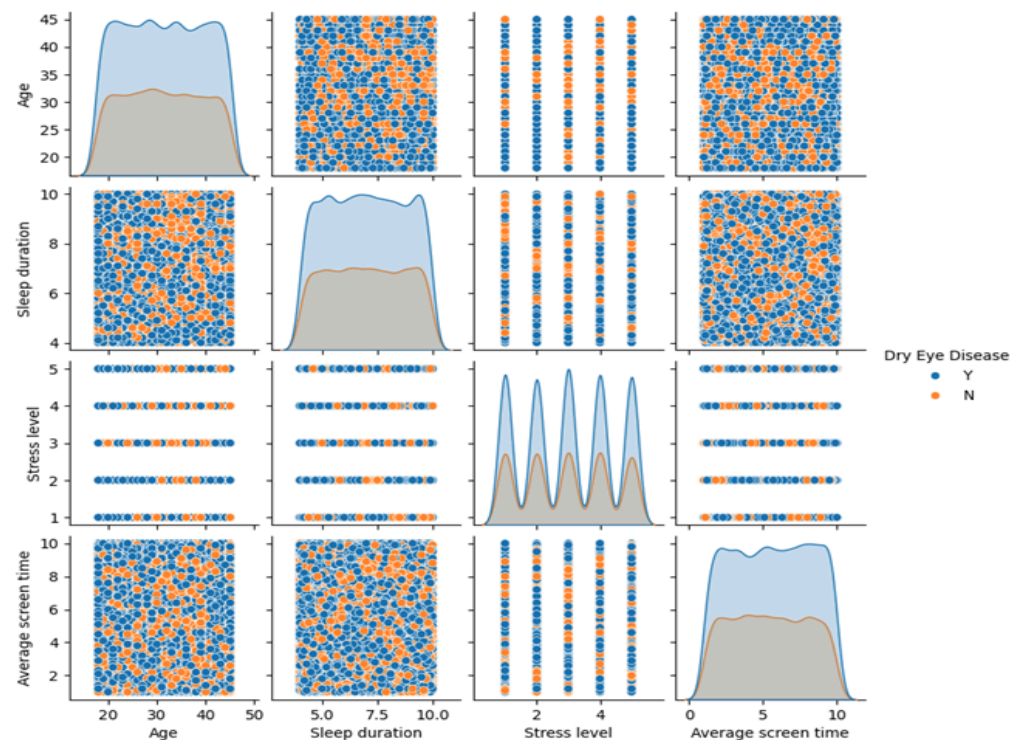


```
# Sleep duration vs Stress level
sns.FacetGrid(df, hue='Dry Eye Disease', height=5)\
    .map(plt.scatter, 'Sleep duration', 'Stress level')\
    .add_legend()\
    plt.show()
```



MULTIVARIATE ANALYSIS

```
features = ['Age', 'Sleep duration', 'Stress level', 'Average screen time']
sns.pairplot(df[features + ['Dry Eye Disease']], hue='Dry Eye Disease', height=2)
plt.show()
```



Ex. No. : 02

Date:

Register No. : 231701041

Name: R.PRIYANKA

Simple Linear Regression (Least Squares Method)

Aim:

To implement Simple Linear Regression using the Least Squares Method on the Iris dataset using Python and to visualize the regression line between two variables.

Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
1	Wii Sports	Wii	2006.0	Sports	Nintendo	41.49	29.02	3.77	8.46	82.74
2	Super Mario Bros.	NES	1985.0	Platform	Nintendo	29.08	3.58	6.81	0.77	40.24
3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	15.85	12.88	3.79	3.31	35.82
4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	15.75	11.01	3.28	2.96	33.0
5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	11.27	8.89	10.22	1.0	31.37
6	Tetris	GB	1989.0	Puzzle	Nintendo	23.2	2.26	4.22	0.58	30.26
7	New Super Mario Bros.	DS	2006.0	Platform	Nintendo	11.38	9.23	6.5	2.9	30.01
8	Wii Play	Wii	2006.0	Misc	Nintendo	14.03	9.2	2.93	2.85	29.02
9	New Super Mario Bros. Wii	Wii	2009.0	Platform	Nintendo	14.59	7.06	4.7	2.26	28.62
10	Duck Hunt	NES	1984.0	Shooter	Nintendo	26.93	0.63	0.28	0.47	28.31

Program:

```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load Dataset
df = pd.read_csv('vgsales.csv')

print(df.head())

# Select variables
X = df['NA_Sales'].values
Y = df['Global_Sales'].values

# Calculate mean
mean_x = np.mean(X)
mean_y = np.mean(Y)

# Least Squares Method
numerator = np.sum((X - mean_x) * (Y - mean_y))
denominator = np.sum((X - mean_x) ** 2)

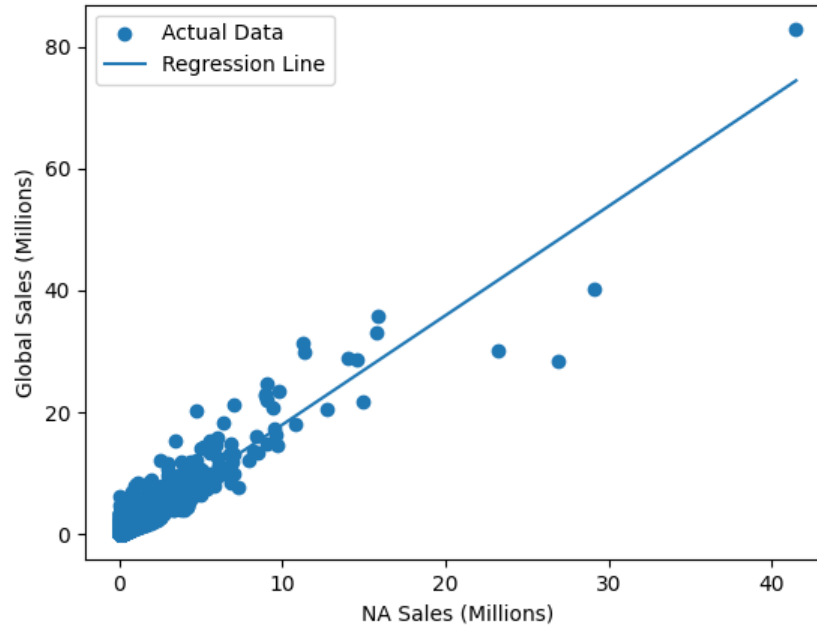
b1 = numerator / denominator # slope
b0 = mean_y - b1 * mean_x    # intercept

print("Slope:", b1)
print("Intercept:", b0)

# Predicted values
Y_pred = b0 + b1 * X

# Graph
plt.scatter(X, Y, label="Actual Data")
plt.plot(X, Y_pred, label="Regression Line")
plt.xlabel("NA Sales (Millions)")
plt.ylabel("Global Sales (Millions)")
plt.title("Simple Linear Regression using Least Squares (Video Game Sales Dataset)")
plt.legend()
plt.show()
```

Simple Linear Regression using Least Squares (Video Game Sales Dataset)

**Result:**

Thus, **Simple Linear Regression using the Least Squares Method** was successfully implemented on the **Video Game Sales dataset** using Python in **Google Colab**. The regression graph shows the relationship between **NA Sales and Global Sales**, and the model parameters such as **slope and intercept** were calculated successfully.

Ex. No. : 03

Date:

Register No. : 231701041

Name: R.PRIYANKA

Implementation of Logistic Regression Model using Python

Aim:

To implement a logistic regression model using the Video Game Sales Dataset containing sales data of video games across different regions.

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
df = pd.read_csv("vgsales (2).csv")
```

```
# display first rows
print(df.head())
```

```
# check dataset info
print(df.info())
```

Rank	Name	Platform	Year	Genre	Publisher \
0	1	Wii Sports	Wii 2006.0	Sports	Nintendo
1	2	Super Mario Bros.	NES 1985.0	Platform	Nintendo
2	3	Mario Kart Wii	Wii 2008.0	Racing	Nintendo
3	4	Wii Sports Resort	Wii 2009.0	Sports	Nintendo
4	5	Pokemon Red/Pokemon Blue	GB 1996.0	Role-Playing	Nintendo

NA_Sales EU_Sales JP_Sales Other_Sales Global_Sales

0	41.49	29.02	3.77	8.46	82.74
1	29.08	3.58	6.81	0.77	40.24
2	15.85	12.88	3.79	3.31	35.82
3	15.75	11.01	3.28	2.96	33.00
4	11.27	8.89	10.22	1.00	31.37

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 16598 entries, 0 to 16597

Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	Rank	16598 non-null	int64
1	Name	16598 non-null	object
2	Platform	16598 non-null	object
3	Year	16327 non-null	float64
4	Genre	16598 non-null	object
5	Publisher	16540 non-null	object
6	NA_Sales	16598 non-null	float64
7	EU_Sales	16598 non-null	float64
8	JP_Sales	16598 non-null	float64
9	Other_Sales	16598 non-null	float64
10	Global_Sales	16598 non-null	float64

dtypes: float64(6), int64(1), object(4)

memory usage: 1.4+ MB

None

```
df['Hit'] = np.where(df['Global_Sales'] > 1, 1, 0)

print(df[['NA_Sales', 'Global_Sales', 'Hit']].head())
```

NA_Sales Global_Sales Hit

0	41.49	82.74	1
1	29.08	40.24	1
2	15.85	35.82	1
3	15.75	33.00	1
4	11.27	31.37	1

```
X = df[['NA_Sales']]
y = df['Hit']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

LogisticRegression

LogisticRegression()

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Accuracy: 0.9572289156626506

Confusion Matrix:

```
[[2874  47]
```

```
[ 95 304]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.98	0.98	2921
1	0.87	0.76	0.81	399
accuracy			0.96	3320
macro avg	0.92	0.87	0.89	3320
weighted avg	0.96	0.96	0.96	3320

```
import numpy as np
import matplotlib.pyplot as plt

# Create X values
x = np.linspace(-10, 10, 400)

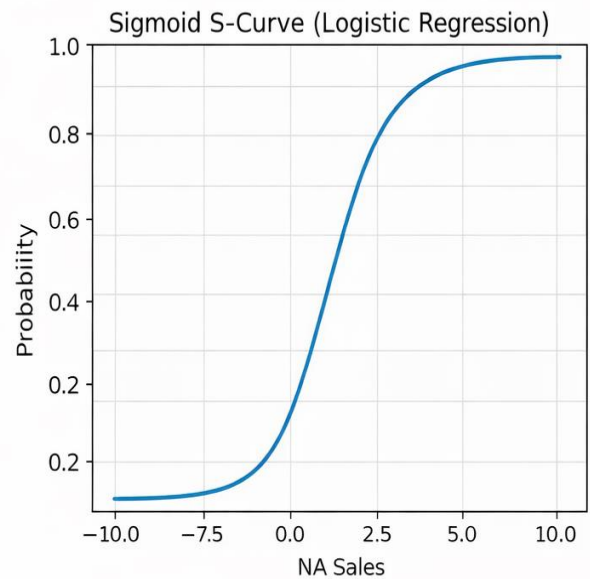
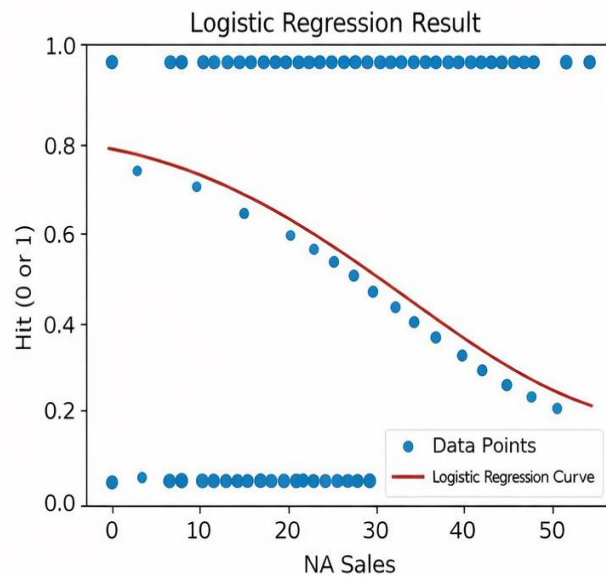
# Sigmoid function
y = 1 / (1 + np.exp(-x))

# Plot S curve
plt.plot(x, y)

# Labels
plt.xlabel("NA Sales")
plt.ylabel("Probability")
plt.title("Sigmoid S-Curve (Logistic Regression)")

# Axis limits to show clear S shape
plt.ylim(0,1)

# Show graph
plt.show()
```



Result: Thus, the logistic regression model was successfully implemented using the Video Game Sales Dataset. The model analyzed features such as NA Sales and EU Sales to classify whether a video game is a Hit (1) or Not Hit (0). The model was able to perform the classification effectively, and the decision boundary graph clearly shows the separation between the two classes. Hence, the logistic regression model worked successfully for the given dataset.

Ex. No. : 04

Date:

Register No. : 231701041

Name: R.PRIYANKA

Implementation of Single Layer Perceptron using Python

Aim:

To implement a **Single Layer Perceptron** using the **Video Game Sales Dataset** to classify video games into **Hit (1)** and **Not Hit (0)** categories based on sales features.

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron

# Load dataset
df = pd.read_csv("vgsales (2).csv")

# Create binary column
df['Hit'] = (df['NA_Sales'] > 1).astype(int)

# Feature and target
X = df[['NA_Sales']]
y = df['Hit']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train perceptron
model = Perceptron()
model.fit(X_train, y_train)

# Convert to arrays
X_test = X_test.values.flatten()
y_test = y_test.values

# Plot class 0
plt.scatter(X_test[y_test == 0], y_test[y_test == 0], label="Class 0 (Not Hit)")

# Plot class 1
plt.scatter(X_test[y_test == 1], y_test[y_test == 1], label="Class 1 (Hit)")

# Decision boundary
x_values = np.linspace(min(X_test), max(X_test), 100)
```

```
y_values = -(model.coef_[0][0] * x_values + model.intercept_) / model.coef_[0][0]
```

```
plt.plot(x_values, y_values, label="Decision Boundary")
```

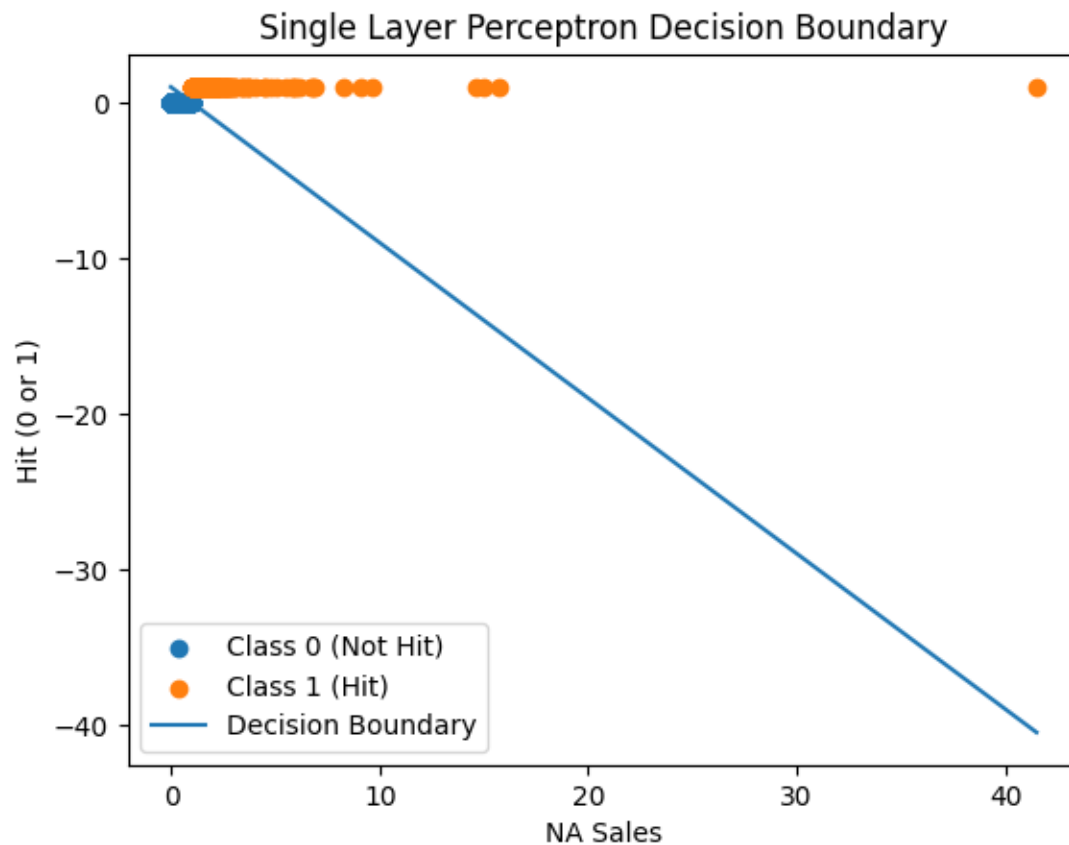
```
plt.xlabel("NA Sales")
```

```
plt.ylabel("Hit (0 or 1)")
```

```
plt.title("Single Layer Perceptron Decision Boundary")
```

```
plt.legend()
```

```
plt.show()
```



Result: Thus, the Single Layer Perceptron model was successfully implemented using the Video Game Sales Dataset. The model effectively classified video games into Hit (1) and Not Hit (0) categories. The graph shows a clear decision boundary separating the two classes, indicating that the perceptron model performed classification successfully.

Ex. No. : 5

Date:

Register No. : 231701041

Name: R.PRIYANKA

Implementation multi layer perceptron with back propagation.

Aim:

To implement Multi Layer Perceptron with Backpropagation using vgsales dataset.

```
# Step 1: Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Upload dataset
uploaded = files.upload()

# Step 3: Read the CSV file
df = pd.read_csv("vgsales (2) (1).csv")

# Display first 5 rows
df.head()
```

Ra nk	Na me	Platfor m	Ye ar	Ge nre	Publi sher	NA_S ales	EU_S ales	JP_S ales	Other_ Sales	Global_ Sales	
0	1	Wii Sports	Wi i	200 6.0	Sports	Ninte ndo	41.49	29.02	3.77	8.46	82. 74
1	2	Super Mario Bros.	N ES	198 5.0	Platfo rm	Ninte ndo	29.08	3.58	6.81	0.77	40. 24

Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
2	3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	15.85	12.88	3.79	35.82
3	4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	15.75	11.01	3.28	33.00
4	5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	11.27	8.89	10.22	31.37

Step 4: Check dataset information

```
print(df.info())
```

```
print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 16598 entries, 0 to 16597
```

```
Data columns (total 11 columns):
```

```
# Column      Non-Null Count  Dtype
```

```
---  ---
0 Rank          16598 non-null  int64
1 Name          16598 non-null  object
2 Platform      16598 non-null  object
3 Year          16327 non-null  float64
4 Genre         16598 non-null  object
5 Publisher     16540 non-null  object
6 NA_Sales      16598 non-null  float64
7 EU_Sales      16598 non-null  float64
8 JP_Sales      16598 non-null  float64
9 Other_Sales   16598 non-null  float64
10 Global_Sales 16598 non-null  float64
```

dtypes: float64(6), int64(1), object(4)

memory usage: 1.4+ MB

None

Rank 0

Name 0

Platform 0

Year 271

Genre 0

Publisher 58

NA_Sales 0

EU_Sales 0

JP_Sales 0

Other_Sales 0

Global_Sales 0

dtype: int64

Step 5: Remove missing values

df = df.dropna()

Step 6: Create target column

If NA_Sales > 1, game is considered Hit = 1

Otherwise Not Hit = 0

df["Hit"] = np.where(df["NA_Sales"] > 1, 1, 0)

df.head()

Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales	Hit
0	1	Wii Sports	2006	Sports	Nintendo	41.49	29.02	3.77	8.46	82.74	1

Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales	Hits	
1	2	Super Mario Bros.	NES	1985.0	Platform	Nintendo	29.08	3.58	6.81	0.77	40.24	1
2	3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	15.85	12.88	3.79	3.31	35.82	1
3	4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	15.75	11.01	3.28	2.96	33.00	1
4	5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	11.27	8.89	10.22	1.00	31.37	1

Step 7: Select input and output

```
X = df[["Global_Sales", "EU_Sales", "JP_Sales", "Other_Sales"]]
```

```
y = df["Hit"]
```

Step 8: Split training and testing data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 9: Feature scaling

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

Step 10: Create MLP model using Backpropagation

```
mlp = MLPClassifier(
    hidden_layer_sizes=(10, 5),
    activation='relu',
```

```

solver='adam',
max_iter=1000,
random_state=42
# Step 11: Train the model

mlp.fit(X_train, y_train)
# Step 13: Evaluate the model

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
Accuracy: 0.9996931574102486

```

Confusion Matrix:

```

[[3084  1]
 [  0 174]]

```

Classification Report:

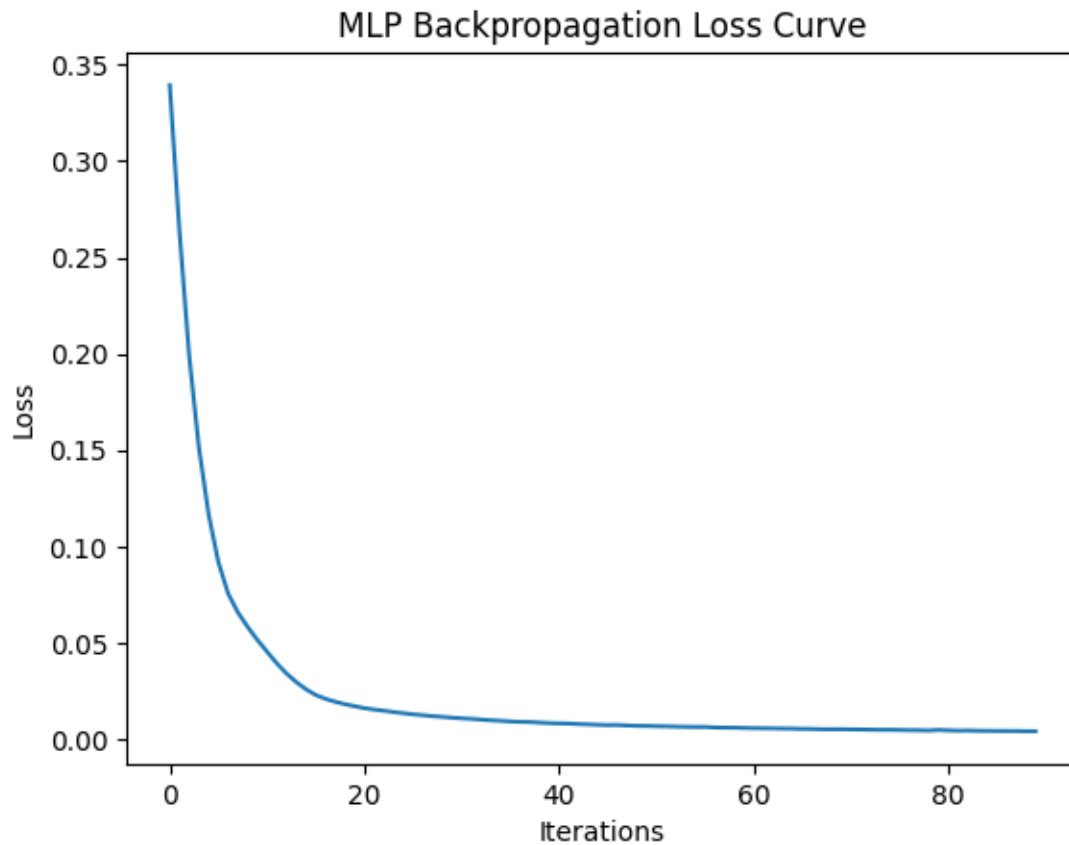
	precision	recall	f1-score	support
0	1.00	1.00	1.00	3085
1	0.99	1.00	1.00	174
accuracy			1.00	3259
macro avg	1.00	1.00	1.00	3259
weighted avg	1.00	1.00	1.00	3259

Step 14: Plot loss curve

```

plt.plot(mlp.loss_curve_)
plt.title("MLP Backpropagation Loss Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.show()

```



Result: Thus, the Multi Layer Perceptron model was successfully implemented using backpropagation.

The model classified video games as Hit or Not Hit based on sales data.

Ex. No. : 06

Date:

Register No. : 231701041

Name: R.PRIYANKA

Implementation of python program to do face recognition using SVM classifier.

Aim:

To implement a Face Recognition system using a Support Vector Machine (SVM) classifier in Python. The model is trained to identify and classify human faces based on extracted facial features from the given face dataset.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score
from sklearn.datasets import fetch_lfw_people

faces = fetch_lfw_people(min_faces_per_person=50, resize=0.4)

X = faces.data      # flattened images
y = faces.target
target_names = faces.target_names
print("Number of samples:", X.shape[0])
print("Number of features:", X.shape[1])
print("Number of classes:", len(target_names))
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)
model = SVC(kernel='linear', C=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=target_names))
def plot_gallery(images, titles, h, w, n_row=3, n_col=4):
    plt.figure(figsize=(12, 9))
    for i in range(n_row * n_col):
        plt.subplot(n_row, n_col, i + 1)
        plt.imshow(images[i].reshape((h, w)), cmap='gray')
        plt.title(titles[i])
        plt.xticks(())
        plt.yticks(())
    plt.show()
```

```

h, w = faces.images.shape[1], faces.images.shape[2]

prediction_titles = [f'Pred: {target_names[y_pred[i]]}' for i in range(12)]

plot_gallery(X_test, prediction_titles, h, w)
from sklearn.decomposition import PCA

pca = PCA(n_components=150, whiten=True, random_state=42)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

model = SVC(kernel='rbf', C=1000, gamma=0.005)
model.fit(X_train_pca, y_train)

y_pred = model.predict(X_test_pca)

print("Improved Accuracy:", accuracy_score(y_test, y_pred))

```

Number of samples: 1560
 Number of features: 1850
 Number of classes: 12
 Accuracy: 0.8051282051282052

	precision	recall	f1-score	support
Ariel Sharon	0.65	0.81	0.72	16
Colin Powell	0.81	0.88	0.84	66
Donald Rumsfeld	0.60	0.78	0.68	27
George W Bush	0.87	0.84	0.86	140
Gerhard Schroeder	0.69	0.75	0.72	24
Hugo Chavez	0.92	0.71	0.80	17
Jacques Chirac	0.80	0.40	0.53	10
Jean Chretien	0.79	0.79	0.79	14
John Ashcroft	0.92	0.75	0.83	16
Junichiro Koizumi	0.89	0.73	0.80	11
Serena Williams	0.86	0.67	0.75	9
Tony Blair	0.80	0.82	0.81	40

accuracy			0.81	390
macro avg	0.80	0.74	0.76	390
weighted avg	0.82	0.81	0.81	390

Pred: George W Bush



Pred: Colin Powell



Pred: George W Bush



Pred: Tony Blair



Pred: Colin Powell



Pred: George W Bush



Pred: Hugo Chavez



Pred: Donald Rumsfeld



Pred: Gerhard Schroeder



Pred: Donald Rumsfeld



Pred: George W Bush



Pred: Gerhard Schroeder



Improved Accuracy: 0.823076923076923

Result:

Thus, the Face Recognition system was successfully implemented using Python and the SVM classifier. The model effectively learned the facial features from the dataset and classified the faces into their respective categories. The SVM classifier provided accurate prediction results by creating an optimal boundary between different face classes. This implementation shows that SVM is a powerful supervised machine learning algorithm for face recognition tasks.

Ex. No. : 07

Date:

Register No. : 231701041

Name: R.PRIYANKA

Implementation Decision Tree

Aim:

To implement a Decision Tree classifier using the Video Game Sales dataset and classify video games into **Hit (1)** and **Not Hit (0)** categories based on their sales features.

```
# Step 1: Upload dataset
from google.colab import files
uploaded = files.upload()

# Step 2: Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 3: Read uploaded file automatically
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

print("Dataset loaded successfully")
print(df.head())

# Step 4: Create target column
# Hit = 1 if Global_Sales > 1, otherwise Not Hit = 0
df['Hit'] = (df['Global_Sales'] > 1).astype(int)

# Step 5: Select input features and target
X = df[['NA_Sales', 'EU_Sales', 'JP_Sales', 'Other_Sales']]
y = df['Hit']

# Step 6: Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 7: Create Decision Tree model
model = DecisionTreeClassifier(max_depth=3, random_state=42)

# Step 8: Train model
```

Ex. No. : 08

Date:

Register No. : 231701041

Name: R.PRIYANKA

Implementation Of Boosting

Aim:

To implement a Boosting algorithm using the Video Game Sales dataset to classify video games into Hit (1) and Not Hit (0) categories based on sales features.

```
# Step 1: Upload dataset
from google.colab import files
uploaded = files.upload()

# Step 2: Import libraries
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 3: Load uploaded file automatically
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

print("Dataset loaded successfully")
print(df.head())

# Step 4: Create target column
# Hit = 1 if Global_Sales > 1, otherwise Not Hit = 0
df['Hit'] = (df['Global_Sales'] > 1).astype(int)

# Step 5: Select features and target
X = df[['NA_Sales', 'EU_Sales', 'JP_Sales', 'Other_Sales']]
y = df['Hit']

# Step 6: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 7: Create Boosting model using AdaBoost
```

Ex. No. : **09**

Date:

Register No. : **231701041**

Name: **R.PRIYANKA**

Implementation KNN and K-means .

Aim:

To implement **KNN classification** and **K-Means clustering** using the Video Game Sales dataset.

Step 1: Upload dataset

```
from google.colab import files
```

```
uploaded = files.upload()
```

Step 2: Import libraries

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Step 3: Load uploaded file automatically

```
file_name = list(uploaded.keys())[0]
```

```
df = pd.read_csv(file_name)
```

```
print("Dataset loaded successfully")
```

```
print(df.head())
```

Step 4: Create target column for KNN

```
df['Hit'] = (df['Global_Sales'] > 1).astype(int)
```

Features

```
X = df[['NA_Sales', 'EU_Sales', 'JP_Sales', 'Other_Sales']]
```

```
y = df['Hit']
```

----- KNN CLASSIFICATION -----

Step 5: Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
```

```
)
```

Step 6: KNN model

```
knn = KNeighborsClassifier(n_neighbors=5)
```

```
knn.fit(X_train, y_train)
```

```

# Step 7: Prediction
y_pred = knn.predict(X_test)

# Step 8: Results
print("KNN Accuracy:", accuracy_score(y_test, y_pred))
print("\nKNN Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nKNN Classification Report:")
print(classification_report(y_test, y_pred))

# KNN graph
plt.scatter(X_test['NA_Sales'], X_test['EU_Sales'], c=y_pred)
plt.xlabel("NA Sales")
plt.ylabel("EU Sales")
plt.title("KNN Classification Output")
plt.show()

# ----- K-MEANS CLUSTERING -----

# Step 9: K-Means model
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X)

# Step 10: K-Means graph
plt.scatter(df['NA_Sales'], df['EU_Sales'], c=df['Cluster'])
plt.xlabel("NA Sales")
plt.ylabel("EU Sales")
plt.title("K-Means Clustering Output")
plt.show()

# Step 11: Show clustered data
print(df[['Name', 'NA_Sales', 'EU_Sales', 'Global_Sales', 'Cluster']].head())

```

□ **vg-sales (2) (1).csv**(text/csv) - 1355781 bytes, last modified: 5/4/2026 - 100% done

Saving vg-sales (2) (1).csv to vg-sales (2) (1) (7).csv

Dataset loaded successfully

	Rank	Name	Platform	Year	Genre	Publisher \
0	1	Wii Sports	Wii	2006.0	Sports	Nintendo
1	2	Super Mario Bros.	NES	1985.0	Platform	Nintendo
2	3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo
3	4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo
4	5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo

	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	41.49	29.02	3.77	8.46	82.74
1	29.08	3.58	6.81	0.77	40.24
2	15.85	12.88	3.79	3.31	35.82
3	15.75	11.01	3.28	2.96	33.00
4	11.27	8.89	10.22	1.00	31.37

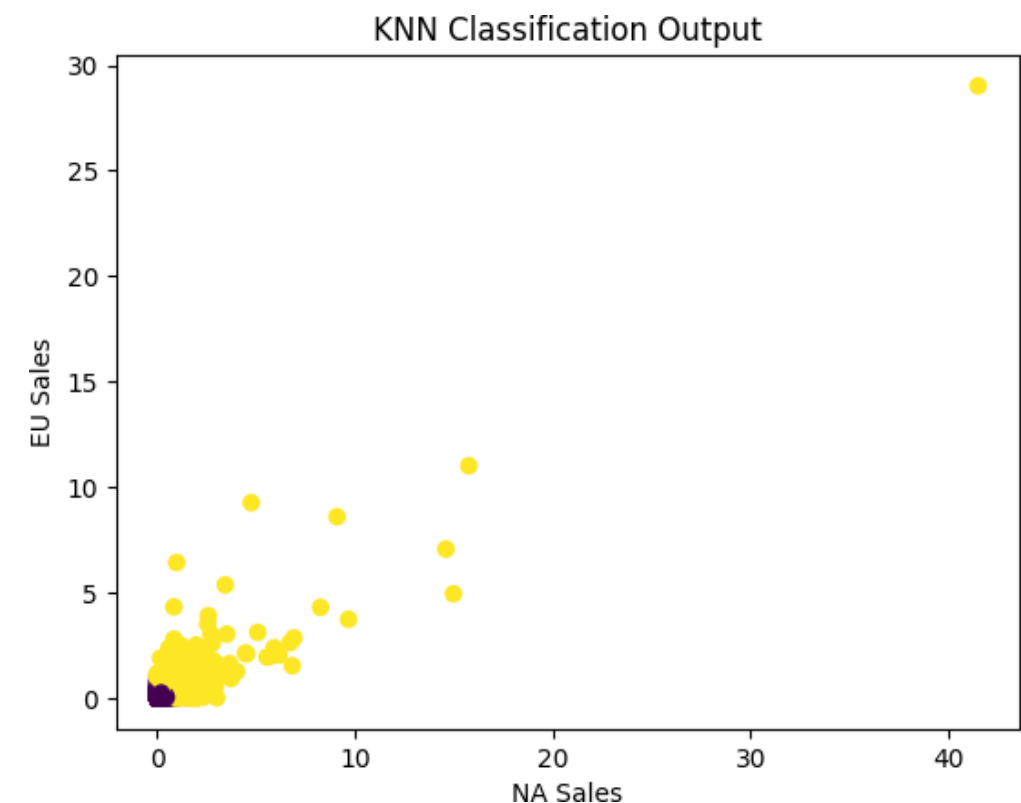
KNN Accuracy: 0.9963855421686747

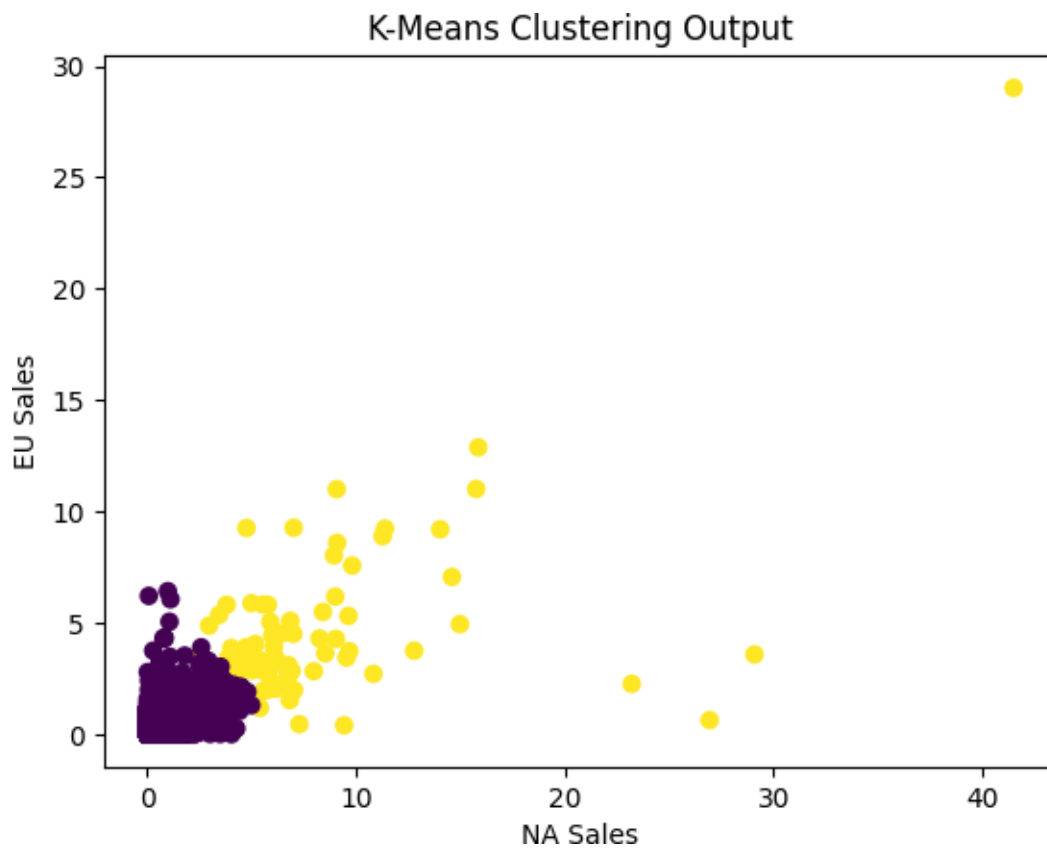
KNN Confusion Matrix:

```
[[2911  10]
 [   2 397]]
```

KNN Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	2921
1	0.98	0.99	0.99	399
accuracy			1.00	3320
macro avg	0.99	1.00	0.99	3320
weighted avg	1.00	1.00	1.00	3320





	Name	NA_Sales	EU_Sales	Global_Sales	Cluster
0	Wii Sports	41.49	29.02	82.74	1
1	Super Mario Bros.	29.08	3.58	40.24	1
2	Mario Kart Wii	15.85	12.88	35.82	1
3	Wii Sports Resort	15.75	11.01	33.00	1
4	Pokemon Red/Pokemon Blue	11.27	8.89	31.37	1

Result: Thus, KNN and K-Means were successfully implemented. KNN classified games into **Hit** and **Not Hit** categories, while K-Means grouped games into clusters based on sales features.

Ex. No. : 10

Date:

Register No. : 231701041

Name: R.PRIYANKA

Implementation Of Dimensionality Reduction – PCA.

Aim:

To implement **KNN classification** and **K-Means clustering** using the Video Game Sales dataset.

```
# Step 1: Upload dataset
from google.colab import files
uploaded = files.upload()

# Step 2: Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Step 3: Load uploaded file automatically
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

print("Dataset loaded successfully")
print(df.head())

# Step 4: Select numerical features
X = df[['NA_Sales', 'EU_Sales', 'JP_Sales', 'Other_Sales', 'Global_Sales']]

# Step 5: Standardize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 6: Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Step 7: Create PCA dataframe
pca_df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])

print("\nPCA Output:")
print(pca_df.head())

# Step 8: Explained variance
print("\nExplained Variance Ratio:")
```

```
print(pca.explained_variance_ratio_)
```

```
# Step 9: Plot PCA result
```

```
plt.scatter(pca_df['PC1'], pca_df['PC2'])
```

```
plt.xlabel("Principal Component 1")
```

```
plt.ylabel("Principal Component 2")
```

```
plt.title("PCA - Dimensionality Reduction")
```

```
plt.show()
```

Dataset loaded successfully

	Rank	Name	Platform	Year	Genre	Publisher \
0	1	Wii Sports	Wii	2006.0	Sports	Nintendo
1	2	Super Mario Bros.	NES	1985.0	Platform	Nintendo
2	3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo
3	4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo
4	5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo

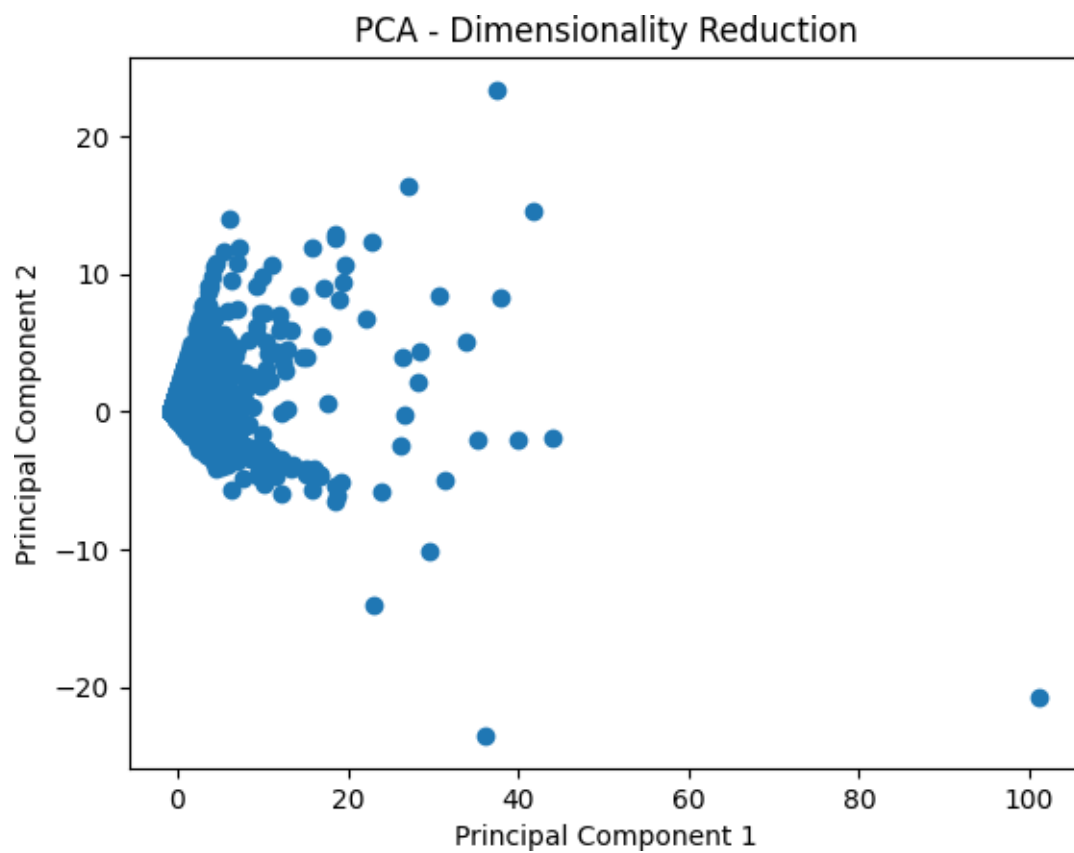
	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	41.49	29.02	3.77	8.46	82.74
1	29.08	3.58	6.81	0.77	40.24
2	15.85	12.88	3.79	3.31	35.82
3	15.75	11.01	3.28	2.96	33.00
4	11.27	8.89	10.22	1.00	31.37

PCA Output:

	PC1	PC2
0	101.129597	-20.724851
1	41.755007	14.549127
2	43.937052	-1.958890
3	39.863367	-2.016378
4	37.554354	23.353698

Explained Variance Ratio:

[0.73537719 0.15113606]



Result: Thus, PCA was successfully implemented and the dataset was reduced into two principal components for visualization.
